

CBCS SCHEME

USN

--	--	--	--	--	--	--	--	--	--

BCS306A

Third Semester B.E./B.Tech. Degree Examination, Dec.2024/Jan.2025

Object Oriented Programming with JAVA

Time: 3 hrs.

Max. Marks: 100

*Note: 1. Answer any FIVE full questions, choosing ONE full question from each module.
2. M : Marks, L: Bloom's level, C: Course outcomes.*

Module - 1			M	L	C
Q.1	a.	List and explain any three features of object oriented programming.	6	L1	CO1
	b.	What do you mean by type conversion and type casting? Give examples.	8	L2	CO1
	c.	How to declare and initialize 1-D and 2-D arrays in Java. Give examples.	6	L2	CO1
OR					
Q.2	a.	List the short circuit operators and show the concept using few examples.	4	L2	CO1
	b.	With a java program, illustrate the use of ternary operator to find the greatest of three numbers.	6	L3	CO1
	c.	Develop a Java program to demonstrate the working of for each version of for loop. Initialize the 2D array with values and print them using for each.	10	L2	CO1
Module - 2					
Q.3	a.	Develop a program in Java to implement a stack of integers.	12	L3	CO2
	b.	What are constructors? Give the types and explain the properties of constructors. Support with appropriate examples.	8	L2	CO2
OR					
Q.4	a.	Illustrate with an example program to pass objects as arguments.	10	L2	CO2
	b.	Explain different access specifiers in Java with example program.	10	L2	CO2
Module - 3					
Q.5	a.	Define inheritance. List and explain different types of inheritance in Java with code snippets.	10	L2	CO3
	b.	Compare and contrast between overloading and overriding in Java with example program for each.	10	L2	CO3
OR					
Q.6	a.	Analyze an interface in Java and list out the speed of an interface. Illustrate with the help of a program the importance of an interface.	10	L2	CO3
	b.	List the different uses of final and demonstrate each with the of code snippets.	10	L2	CO3

Module – 4

Q.7	a.	Define a package. Explain how to create user defined package with example.	7	L2	CO4
	b.	Discuss about exception handling in Java. Give the framework of the exception handling block. List the types of exception.	8	L2	CO4
	c.	Develop a Java program to raise a custom exception for division by zero using try, catch, throw and finally.	5	L3	CO4

OR

Q.8	a.	Compare throw and throws keyword by providing suitable example program.	10	L2	CO4
	b.	Explain about the need for finally block.	5	L2	CO4
	c.	Discuss about chained exceptions.	5	L2	CO4

Module – 5

Q.9	a.	Define thread. Demonstrate creation of multiple threads with a program.	10	L2	CO5
	b.	Explain the two ways in which Java threads can be instantiated. Support your explanation with a sample program.	10	L2	CO5

OR

Q.10	a.	What is enumeration? Explain the methods values() and valueOf().	10	L2	CO5
	b.	Explain about type wrappers and auto boxing.	10	L2	CO5

Object Oriented Programming with Java.

Sem - 3rd

Subj Code: BCS 306A.

1a) Object - Oriented Programming (oop)

Java is an object-oriented programming language that supports the principles of oop. Including encapsulation, inheritance & polymorphism.

Robust Security :- Java's security features include memory management, data typing & exception handling, which help prevent common programming errors that can lead to security vulnerabilities.

Platform Independence :- It means that java programs can run on any device that has Java virtual machine installed.

1b) Type casting :- It is a method or process that converts a data types into another data type in both ways manually & automatically.

* Widening Casting :- It takes place when the two types are compatible the target type is larger than the source type.

byte $\rightarrow$ short $\rightarrow$ int $\rightarrow$ long $\rightarrow$ float $\rightarrow$ double

Narrowing:- When we are assigning a larger type value to a variable of smaller type then we need to perform explicit type casting.

double $\rightarrow$ float $\rightarrow$ long $\rightarrow$ int $\rightarrow$ short $\rightarrow$ byte

eg:- class siml

```
{  
    public static void main(String[] args)
```

```
{  
    int i = 100;
```

```
    float f1 = i;
```

```
    System.out.println(f1);
```

```
    System.out.println(f1);
```

```
    double lgh = 323.142
```

```
    int low = (int) lgh;
```

```
    S.o.p(low);
```

```
}  
}
```

1.6.) An array is a group of like-typed variables that are referred to by a common name

1-D:- We can create an array by using

datatype arrayname[] = new datatype [Array size]

eg:- int a[] = new int [5];

Processing Arrays

Class Array1

```
public static void main(String[] args)
```

```
{  
    double[] mylist = {10, 20, 30, 40, 50}
```

```
    for (int i = 0; i < 4; i++)
```

```
    {  
        s.op(mylist[i]);
```

```
    }  
}
```

2D-array: - It is an array of array.

```
int twoD[][] = new int[3][3];
```

class twoD

```
{  
    public static void main(String[] args)
```

```
{  
    int twoD[][] = new int[3][3];
```

```
    for (int i = 0; i < 3; i++)
```

```
    {  
        for (int j = 0; j < 3; j++)
```

```
        {  
            s.op(twoD[i][j]);
```

```
        }  
    }  
}
```


2a) Short circuit operators

These are logical operators that evaluate the second operand only.

1) Logical AND (&&)

2) Logical OR (||)

→ Evaluates the second operand only if the first operand is true.

→ Returns true if both operands are true otherwise returns false.

```
boolean a = true;
```

```
boolean b = false;
```

```
boolean res = a && b;
```

```
res = false;
```

⇒ Logical OR :- Evaluates the second operand only if the first operand is false

```
boolean a = false;
```

```
boolean b = true;
```

```
boolean res = a || b;
```

```
res = true;
```

2b) Ternary operator :- Also known as

Conditional operator. It's concise way to express simple if-else statement in a single line of code.

Condition ? value(true) : value(false)

class main

```
{  
    public static void main (String[] args)
```

```
{  
    int a=10, b=20, c=30;
```

```
    int grt = (a > b) ? (a > c ? a : c) : (b > c ? b : c);
```

```
    s.o.p (grt);
```

```
}  
}
```

2c.) It is also known as enhanced for loop, its concise way to iterate over arrays, collections & other iterable objects

```
for (type var : iterable)
```

```
{  
    // code  
}
```

eg:- class main

```
{  
    public static void main (String[] args)
```

```
{  
    String[] colors = {"red", "green", "blue"};
```

```
    for (String clr : colors)
```

```
    {  
        s.o.p (clr);
```

```
    }
```

```
}  
}
```


Class Main

```
{  
    public static void main(String[] args)
```

```
{  
    int aD[][] = { { 1, 2, 3, 4 },  
                   { 5, 6, 7, 8 },  
                   { 9, 10, 11, 12 }  
                };
```

```
    for (int[] row : aD)
```

```
    {  
        for (int ele : row)
```

```
        {  
            s.o.p(ele);
```

```
        }  
    }  
}
```

3a.) Class Stack

```
{  
    static int a[] = new int[10];
```

```
    static int ch, n, top = -1, item, i;
```

```
    public static void push()
```

```
    {  
        if (top >= n-1)
```

```
        {  
            s.o.p("stk overflow");
```

```
        }  
    }  
    else
```

```
    {  
        item = sc.nextInt();
```

```
        top++;
```

```
        a[top] = item;
```

```
    }  
}
```



```
public static void pop()
```

```
{ if (top == -1)
```

```
{ s.o.p ("stack is underflow");
```

```
    }
```

```
else
```

```
{ s.o.p ("deleted value is " + a[top]);
```

```
    top--;
```

```
    }
```

```
public static void display()
```

```
{ if (top == -1)
```

```
{ s.o.p ("stack is empty");
```

```
    }
```

```
else
```

```
{ for (int i = top; i >= 0; i--)
```

```
{ s.o.p (a[i]);
```

```
    }
```

3b) Constructors :- These are special methods used to initialize objects.

Types :-

- 1) No-Arg Constructor
- 2) Parameterized
- 3) Default Constructor.

No-Arg :- If a constructor does not accept any parameters it is called "no-argument".

class A1

```
{ A1()
```

```
    }
```


2) Java Parameterized Constructor

→ It can also accept one or more parameters.
Such constructors are known as parameterized constructors.

```
class cse
{
    int abc;
    cse (int a)
    {
        abc = a;
    }
    public static void main (String args[])
    {
        cse cl = new cse (20);
        S.o.p (cl.abc);
    }
}
```

3) Java Default Constructor : - If we do not create any constructor the Java compiler automatically creates a no-arg constructor during the execution of the program.

```
class cse
{
    int abc = 10;
    public static void main (String args[])
    {
        cse mca = new cse ();
        S.o.p (mca.abc);
    }
}
```


14a.) Object as a Parameter :- In this approach a reference to an argument is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call.

Changes made to the parameter will affect the argument used to call the subroutine.

class a1

```
{
    String name;
    a1(String name)
    {
        this.name = name;
    }
}
```

void disp(a2 s)

```
{
    s.name = "mech";
}
```

```
{
}
```

class a2

```
{
    psvm(String[] args)
    {
        a2 s = new a2("cse");
        s.disp(s);
    }
}
```

When parameter is call by reference the caller & callee operate on same object.

Any changes to instance variable it will change to original value.

4b) Access Specifiers

- Public :- The member is accessible from any other class.
- Protected: The member is accessible within its own package and by subclasses
- Default (no modifier): The member is accessible only within its own package
- Private: The member is accessible only within its own class.

Class cse.

```
{  
    public int num = 66;  
    void display()  
    {  
        s.o.p("this is public");  
    }  
}
```

Class aml

```
{  
    public static void main (String[] args)  
    {  
        cse any = new cse();  
        s.o.p(any.num);  
        cse.display();  
    }  
}
```


30) The process of obtaining the data members and methods from one class to another class by using the extends keyword.

Types of inheritance

- Single level
- Multi-level.

The class which is give data members & methods is known as base class

The class which is taking data members & ~~method~~ methods is known as child class.

Class Main

```
{ int boys = 10;
```

```
}
```

Class cse extends Main

```
{ int girls = 20;
```

```
public static void main (String args[])
```

```
{
```

```
    cse p = new cse();
```

```
    s.o.p (p. boys)
```

```
    s.o.p (p. girls);
```

```
}
```

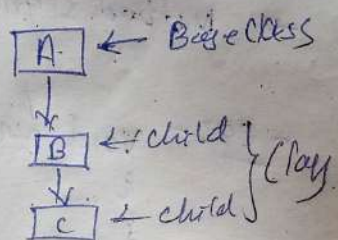
* Multi-level : - It inherits a class which inherits another class.

Class A

```
{ void funA()
```

```
{ s.o.p ("A");
```

```
}
```



Class B extends A

```
{  
  void funB()  
  {  
    s.o.p("B");  
  }  
}
```

Class C extends B

```
{  
  void func()  
  {  
    s.o.p("C");  
  }  
}
```

Class Main

```
{  
  psvm (String[] args)  
  {  
    C obj = new C();  
    obj.funA();  
    obj.funB();  
    obj.func();  
  }  
}
```

5B)

overloading

multiple methods with same name

→ compile-time polymorphism

→ Method signature

eg:-
void func()
{
 ...
}

void fun(int a, int b);
{
 ...
}

overriding

- Subclass provides different implementation

- Runtime polymorphism

- Method signature & return type must be same as superclass.

class Main

```
{  
    void sub()  
    {  
        s.o.p("java");  
    }  
    void sub(int a)  
    {  
        s.o.p(a);  
    }  
}
```

```
class CSE  
{  
    public static void main(String[] args){  
        Main m = new Main();  
        m.C.sub();  
        C.sub(10);  
    }  
}
```

class car

```
{  
    void run()  
    {  
        s.o.p("car is running");  
    }  
}
```

class bike extends car

```
{  
    void run()  
    {  
        s.o.p("bike is running");  
    }  
}
```

class adal

```
{  
    psum(String[] args)  
    {  
        bike d = new bike();  
        d.run();  
    }  
}
```

6a) Interface is a mechanism to achieve abstraction & multiple inheritances.

→ Interface is a keyword

→ All the methods in an interface are declared with an empty body. & are public and all fields are public, static, final by default.

interface name

{

// declare variables

// declare method of abstract

}

eg1 - interface socket

{ public void laptop();

public void mobile();

}

Class room implements socket

{

public void laptop()

{ s.o.p("laptop is charging");

}

Class cse

{

psvm (String[] args)

{

room h1 = new room();

h1.laptop();

}

}

6b) Using final to prevent overriding :-
Method's declared as final cannot be overridden.

Class A

```
{ final void disp()  
  { S.o.p ("class A");  
}
```

Class B extends A

```
{ void display()  
  { S.o.p ("class A");  
}
```

here we can't override display() because it is declared as final in parent class.

Using final to prevent inheritance final
with class from being inherited

final class A

```
{  
  // -  
}
```

Class B extends A //error! can't

inherit class A.

```
{  
}
```

7a) Package can be defined as grouping of related types (classes, interface).
We can create a package by adding package command at the start of java file.

Syntax:-

```
package mypack;
```

We can create hierarchy of packages by separating names with dot.

eg:- package pkg1, pkg2, pkg3;

Step 1:- Create a package

Step 2:- Compile the package

```
javac mypackage/hello.java
```

Step 3:- use the package in another class

```
import mypackage.Hello;
```

```
public class main
```

```
{  
    psvm(String[] args)
```

```
{  
    Hello hello = new Hello();
```

```
    hello.sayHello();  
}
```

```
}
```


7b) Exception is a run-time error that occurs during the execution of a java program.

Techniques:-

- try
- catch
- throw
- throws
- finally

→ try Block:- It might throw an exception must be followed by catch or finally

- Catch:- Exceptions thrown during execution of the try block can be caught.

- finally:- It is always executed.

- throw:- Used to explicitly throw an exception from a method.

- throws:- Used to declare that a method might throw an exception.

Class cse

```
{ psum (String[] args)
```

```
{
```

```
    try
```

```
    {
```

```
        int data = 100/0;
```

```
    }
```

```
    catch (Exception e)
```

```
    {
```

```
        s.o.p(e);
```

```
    }
```

```
    finally
```

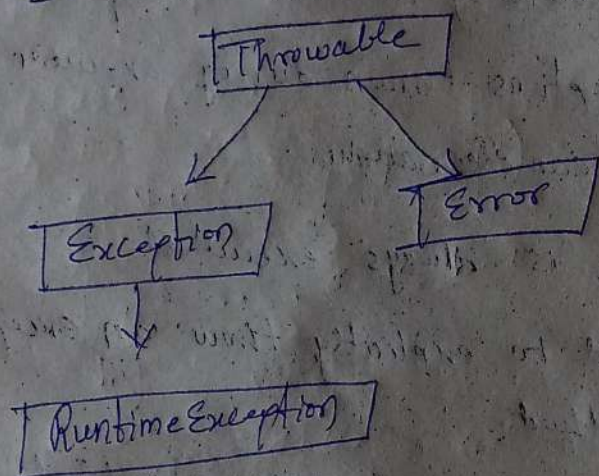
```
    {
```

```
        s.o.p ("rest of the code");
```

```
    }
```

Exception Types

- All exception types are subclasses of the built-in class Throwable.
- Throwable are two subclasses that partition exceptions into two distinct branches
- Subclass of Exception is called RuntimeException
- Errors are used by the Java run-time system to indicate errors.



7C.)

```
class myclass
{
    public divide(String msg)
    {
        super(msg);
    }
}
```

```
class main
{
    public divide(int a, int b)
    {
        if (b == 0)
        {
            throw new divide("Error");
        }
    }
}
```



```

    return a/b;
}

public static void main(String[] args) {
    int res = 0;

    try {
        res = divide(10, 0);
    } catch (Exception e) {
        s.op(e);
    } finally {
        s.op(res);
    }
}
}

```

8a)

Throw

Used to throw an exception within method body or code block.

Syntax :-

throw new ExceptionType("msg");

Used to create & throw an exception

Cannot throw multiple exceptions

Throws

Declares in method

In method signature

Syntax :-

returnType methodName
() throws Exception

Used to inform callers about possible exceptions

can declare multiple exceptions.

- 8b) Finally keyword is used to execute code along with try or catch statements no matter if there is an exception or not.
- It will execute whether or not an exception is thrown. If an exception is thrown the finally block will execute even if no catch statements matching the exception.
- It is an optional.

```
class Main
```

```
{  
    psum (String[] args)
```

```
{  
    int res, a=10, b=0;
```

```
    try
```

```
{
```

```
        res = a/b;
```

```
        S.o.p (res);
```

```
    }  
    catch (Exception e)
```

```
{  
        S.o.p (e);
```

```
    }
```

```
    finally
```

```
{
```

```
        S.o.p ("finally block executed");
```

```
    }
```

```
}
```

```
{
```


8c) Chained exceptions :- These are the exceptions which occur one after another for most of the time to response to an exception are given by an application by throwing another exception.

It is also known as "nesting exception" which occurs one after the other.

The constructors that support chained exceptions

Throwable (Throwable causeExc)

Throwable (String msg, Throwable causeExc)

Methods

Throwable get cause()

Throwable init cause(Throwable causeExc)

9a) It is designed to efficiently handle multiple tasks at once, improving performance and responsiveness.

Class ABC implements Runnable

```
{  
    public void run()
```

```
{  
        S.o.p (Thread.currentThread().getId());
```

```
}
```

public class Main

```
{  
    psum (String[] args)
```



```
Runnable task = new MyRunnable()
```

```
Runnable task1 = new ABC();
```

```
Thread t1 = new Thread(task1)
```

```
Thread t2 = new Thread(task1)
```

```
t1.start();
```

```
t2.start();
```

```
}
```

```
}
```

9b) Creating a Thread

→ Implementing the Runnable Interface

1) Create a class that implements the Runnable Interface.

2) This class must override the run() method which contains the code that will be executed in the new thread.

3) Create an instance of Thread class passing an instance of our Runnable implementation to its constructor.

Class ABC implements Runnable

```
1 public void run()
```

```
2 { s.o.p("thread is running");
```

```
3 }
```


class aml

```
{
    public static void main (String[] args)
    {
        ABC mr = new ABC();
        Thread t1 = new Thread (mr);
        t1.start();
    }
}
```

Extends:- When we extend the thread class we need to override the run() method to specify the task to be performed by the thread.

start() method is called to begin execution which is internally calls run()

eg:- class mythread extends Thread

```
{
    public void run()
    {
        S.o.p("thread is running");
    }
}
```

class aml

```
{
    psum (String[] args)
    {
        mythread t = new mythread();
        t.start();
    }
}
```

10a) Enumerations :- It is list of named fixed constants. And It is defined using the enum keyword.

values() :- It returns an array of all the constants of the enum type in the order they are declared

```
enum sub { java, maths, sea; }  
class abc
```

```
{  
    psvm (String[] args)  
    {  
        sub[] td = sub.values();  
        for (sub day : td)  
        {  
            s.o.p (day);  
        }  
    }  
}
```

valueOf() :- It returns the enum constant of the specified name

```
enum sub { java, maths, sea; }  
class abc
```

```
{  
    psvm (String[] args)  
    {  
        sub td = sub.valueOf("sea");  
        s.o.p (td);  
    }  
}
```


10b) Type wrappers :-

In java each primitive data type has a corresponding wrapper class that allows primitives to be used as objects.

Boolean :-

Primitive Type: boolean

Wrapper class: Boolean

Byte :-

Primitive - byte

Wrapper → Byte

Short :-

primitive → short

Wrapper → Short

Integer :-

primitive :- char

Wrapper :- Character

* Autoboxing :-

It is the automatic conversion of primitive type to its corresponding wrapper class.

class cse

```
{  
    public static void main(String args[])  
    {
```

```
        Integer obj = 100;
```

```
        int i = obj;
```

```
        System.out.println(i);
```

```
    }  
}
```

Print

P. Patil
HOD

CSE (AI & ML)
KLS Vishwanathrao Deshpande
Institute of Technology, Halliyat